

# LAB 6: Object-relational mapping, SOLID principles and Design pattern

## I. Object-relational mapping (ORM)

ORM stands for **Object-Relational Mapping**.

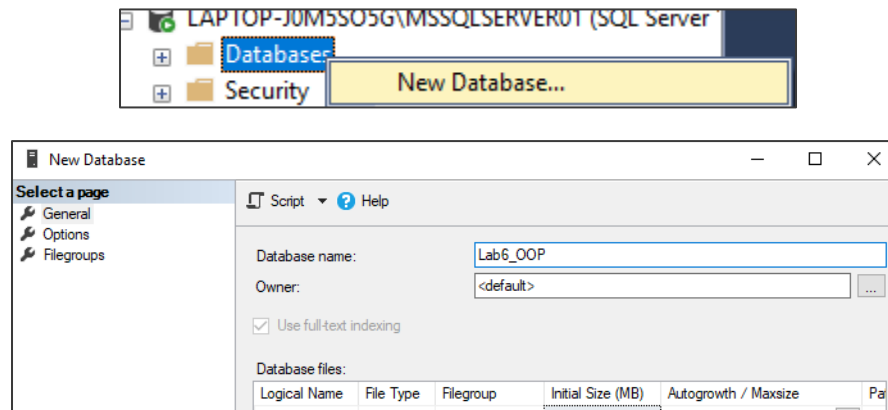
It is a programming technique that **allows you to interact with a relational database using the programming language's objects**, instead of writing raw SQL queries.

**How does ORM work?**

- It **maps database tables to classes** in your code.
- It maps **table rows to instances (objects)** of those classes.
- It automatically converts your object operations (create, read, update, delete) into SQL queries behind the scenes.

**Interacting with the database using raw SQL**

1. Go to SQL Server and create a database named "Lab6\_OOP":



2. Check which port SQL Server is currently running on:

```
use master
go
xp_readerrorlog 0,1, N'Server is listening on'
GO
```

| 144 %            |                         |             |                                                   |
|------------------|-------------------------|-------------|---------------------------------------------------|
| Results Messages |                         |             |                                                   |
|                  | LogDate                 | ProcessInfo | Text                                              |
| 1                | 2025-05-16 13:12:15.650 | Server      | Server is listening on [ ::1 <ipv6> 50075].       |
| 2                | 2025-05-16 13:12:15.650 | Server      | Server is listening on [ 127.0.0.1 <ipv4> 50075]. |

Create table “User”:

```
CREATE TABLE users (
  id INT PRIMARY KEY,
  name VARCHAR(100)
);
```

3. Go to the following path to download the JDBC Driver for SQL Server:

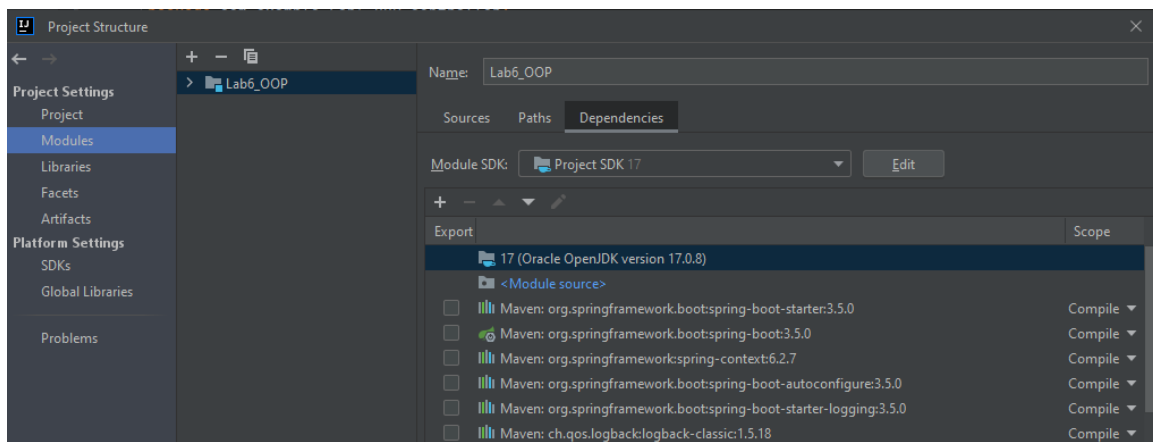
<https://learn.microsoft.com/en-us/sql/connect/jdbc/download-microsoft-jdbc-driver-for-sql-server?view=sql-server-ver17>

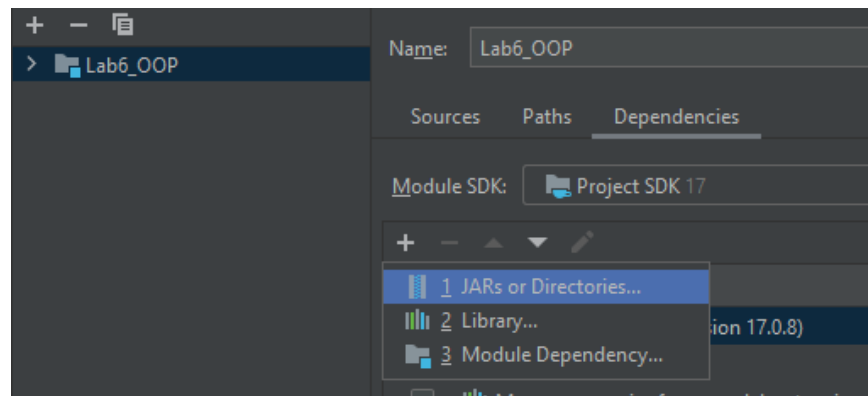
4. Extract the folder you just downloaded:

| Downloads > sqljdbc_12.10.0.0_enu > sqljdbc_12.10 > enu > jars |                  |                     |          |  |
|----------------------------------------------------------------|------------------|---------------------|----------|--|
| Name                                                           | Date modified    | Type                | Size     |  |
| mssql-jdbc-12.10.0.jre8.jar                                    | 3/7/2025 4:47 AM | Executable Jar File | 1,451 KB |  |
| mssql-jdbc-12.10.0.jre8-javadoc.jar                            | 3/7/2025 4:47 AM | Executable Jar File | 1,250 KB |  |
| mssql-jdbc-12.10.0.jre8-sources.jar                            | 3/7/2025 4:47 AM | Executable Jar File | 981 KB   |  |
| mssql-jdbc-12.10.0.jre11.jar                                   | 3/7/2025 4:47 AM | Executable Jar File | 1,464 KB |  |
| mssql-jdbc-12.10.0.jre11-javadoc.jar                           | 3/7/2025 4:47 AM | Executable Jar File | 1,272 KB |  |
| mssql-jdbc-12.10.0.jre11-sources.jar                           | 3/7/2025 4:47 AM | Executable Jar File | 981 KB   |  |

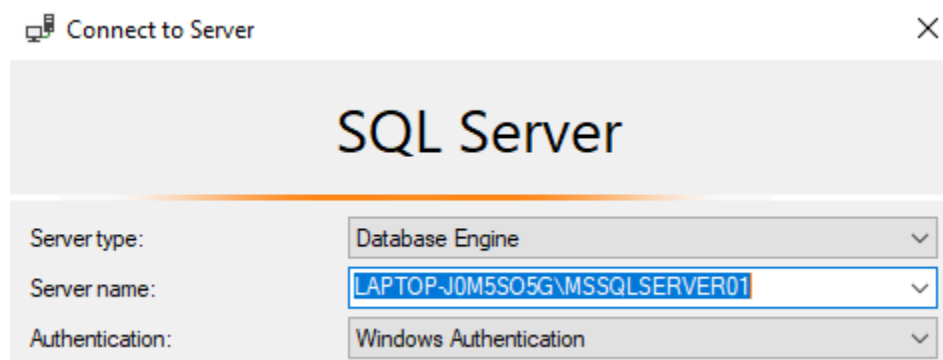
5. In IntelliJ, add the necessary dependencies from the extracted folder by doing the following:

File → Project Structure → Modules → Dependencies → + JARs or directories

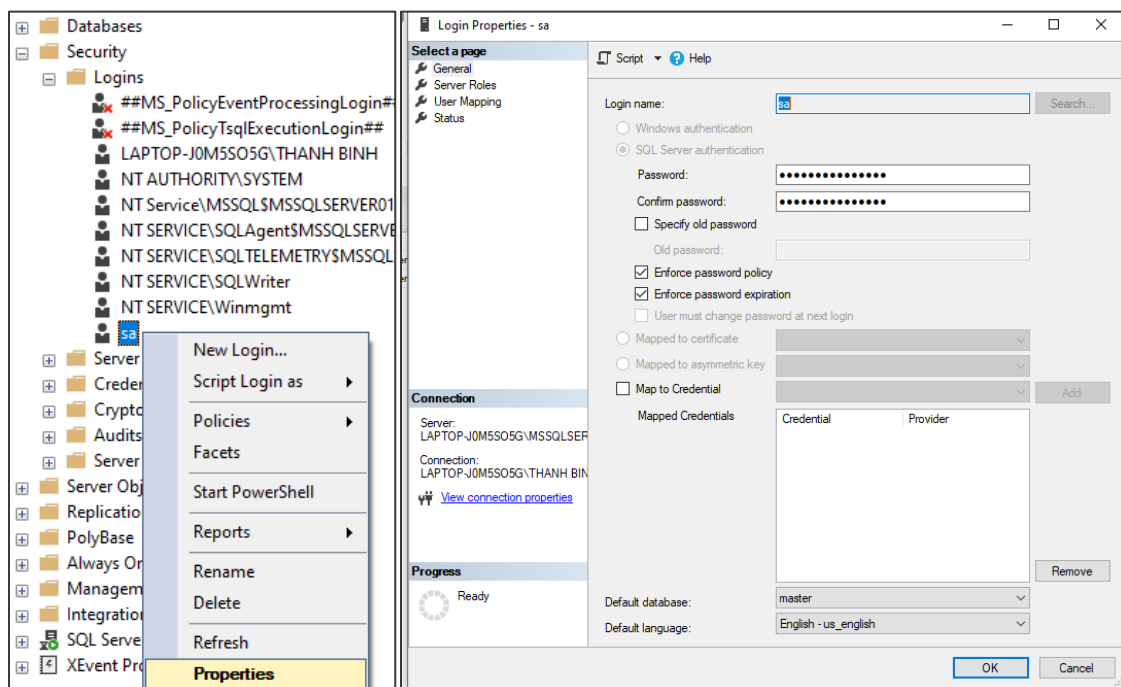




6. Log into SQL Server using "Windows Authentication":



7. Go to the "Security" section, select "sa", and set a password for the "sa" user:



8. In the main code file as shown, you can interact directly with the database using Java code:

```
import java.sql.*;

public class RawSQLException {
    private static final String URL = "jdbc:sqlserver://localhost:50075;databaseName=Lab6_OOP;encrypt=true;trustServerCertificate=true";
    private static final String USER = "sa";
    private static final String PASS = "123456";

    public static void main(String[] args) {
        try (Connection conn = DriverManager.getConnection(URL, USER, PASS)) {
            // Insert
            String insertSQL = "INSERT INTO users (id, name) VALUES (1, 'Alice')";
            conn.createStatement().executeUpdate(insertSQL);

            // Update
            String updateSQL = "UPDATE users SET name = 'Alice Updated' WHERE id = 1";
            conn.createStatement().executeUpdate(updateSQL);

            // Delete
            String deleteSQL = "DELETE FROM users WHERE id = 1";
            conn.createStatement().executeUpdate(deleteSQL);

            System.out.println("Raw SQL: Insert, Update, Delete done.");
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

private static final String *URL* =

"jdbc:sqlserver://localhost:50075;databaseName=Lab6\_OOP;encrypt=true;trustServerCertificate=true";

Result:

```
Raw SQL: Insert, Update, Delete done.
```

| Results |       | Messages |               |
|---------|-------|----------|---------------|
| id      | name  | id       | name          |
| 1       | Alice | 1        | Alice Updated |

## Manual ORM-Style (Object-Oriented JDBC)

1. Create a "User" class representing a user, as shown below:

```
public class User {
    private int id;
    private String name;

    public User() {
    }

    public User(int id, String name) {
        this.id = id;
        this.name = name;
    }

    // Getters and setters
    public int getId() { return id; }

    public void setId(int id) { this.id = id; }

    public String getName() { return name; }

    public void setName(String name) { this.name = name; }
}
```

2. Create a "UserDAO" class — this class helps us interact with the database using "PreparedStatement" and input parameters.

```
public class UserDAO {
    private Connection conn;

    public UserDAO(Connection conn) { this.conn = conn; }

    public void insert(User user) throws SQLException {
        String sql = "INSERT INTO users (id, name) VALUES (?, ?)";
        try (PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setInt(1, user.getId());
            stmt.setString(2, user.getName());
            stmt.executeUpdate();
        }
    }

    public void update(User user) throws SQLException {
        String sql = "UPDATE users SET name = ? WHERE id = ?";
        try (PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, user.getName());
            stmt.setInt(2, user.getId());
            stmt.executeUpdate();
        }
    }

    public void delete(int id) throws SQLException {
        String sql = "DELETE FROM users WHERE id = ?";
        try (PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setInt(1, id);
            stmt.executeUpdate();
        }
    }
}
```

3. At this point, in the main method, we will only interact with objects instead of SQL statements as in the previous part.

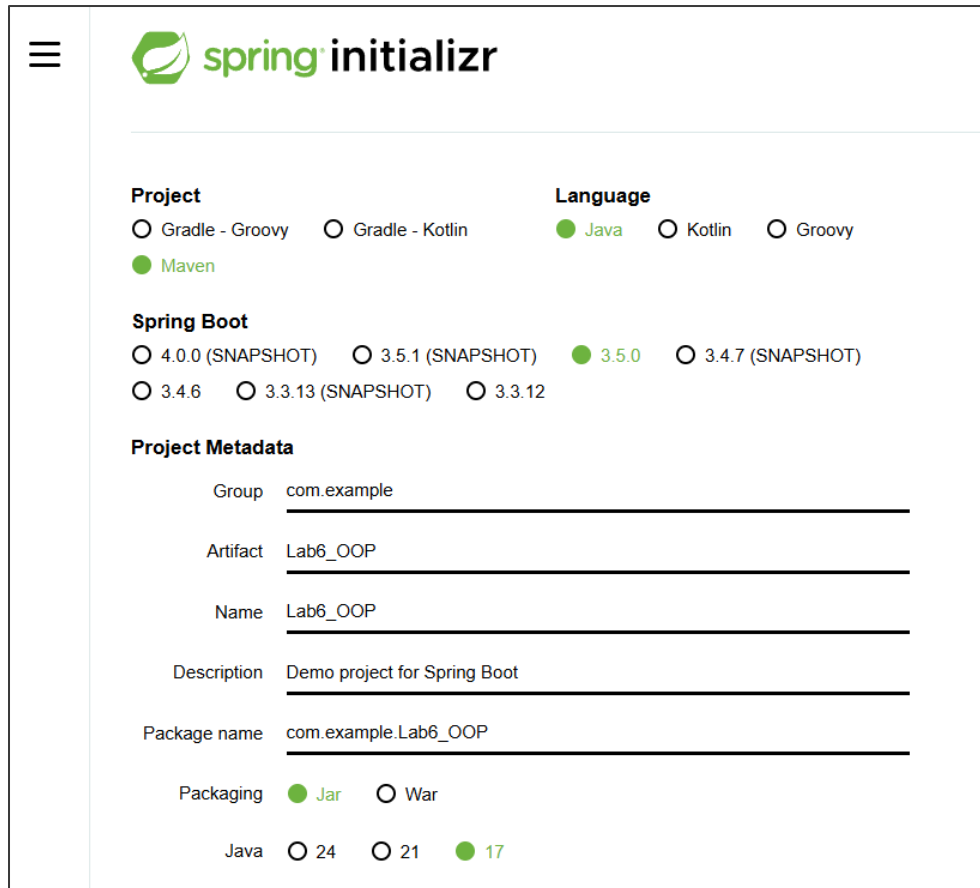
```
ORMStyleExample.java
1  import java.sql.*;
2
3  public class ORMStyleExample {
4      private static final String URL = "jdbc:sqlserver://localhost:50075;databaseName=Lab6_00P;encrypt=true;trustServerCertificate=true";
5      private static final String USER = "sa";
6      private static final String PASS = "123456";
7
8      public static void main(String[] args) {
9          try (Connection conn = DriverManager.getConnection(URL, USER, PASS)) {
10              UserDao userDao = new UserDao(conn);
11
12              // Insert
13              User user = new User(id: 2, name: "Bob");
14              userDao.insert(user);
15              System.out.println("Inserted: " + user.getName());
16
17              // Update
18              user.setName("Bob Updated");
19              userDao.update(user);
20              System.out.println("Updated: " + user.getName());
21
22              // Delete
23              userDao.delete(user.getId());
24              System.out.println("Deleted user with ID: " + user.getId());
25
26          } catch (SQLException e) {
27              e.printStackTrace();
28          }
29      }
30  }
```

Result:

```
Inserted: Bob
Updated: Bob Updated
```

## ORM using the Spring Boot framework

1. Go to the website <https://start.spring.io/> and select the configurations as shown below:



The screenshot shows the Spring Initializr web interface. On the left is a hamburger menu icon. The main content area has the 'spring initializr' logo at the top. Below the logo, there are three sections of configuration options:

- Project**: Radio buttons for 'Gradle - Groovy', 'Gradle - Kotlin', 'Java' (selected), 'Kotlin', and 'Groovy'. Below these is a radio button for 'Maven' (selected).
- Spring Boot**: Radio buttons for '4.0.0 (SNAPSHOT)', '3.5.1 (SNAPSHOT)', '3.5.0' (selected), '3.4.7 (SNAPSHOT)', '3.4.6', '3.3.13 (SNAPSHOT)', and '3.3.12'.
- Project Metadata**: A series of text input fields for:
  - Group: com.example
  - Artifact: Lab6\_OOP
  - Name: Lab6\_OOP
  - Description: Demo project for Spring Boot
  - Package name: com.example.Lab6\_OOP

Below the metadata fields, there are more options:

- Packaging**: Radio buttons for 'Jar' (selected) and 'War'.
- Java**: Radio buttons for '24', '21', and '17' (selected).

2. Download and extract the files:



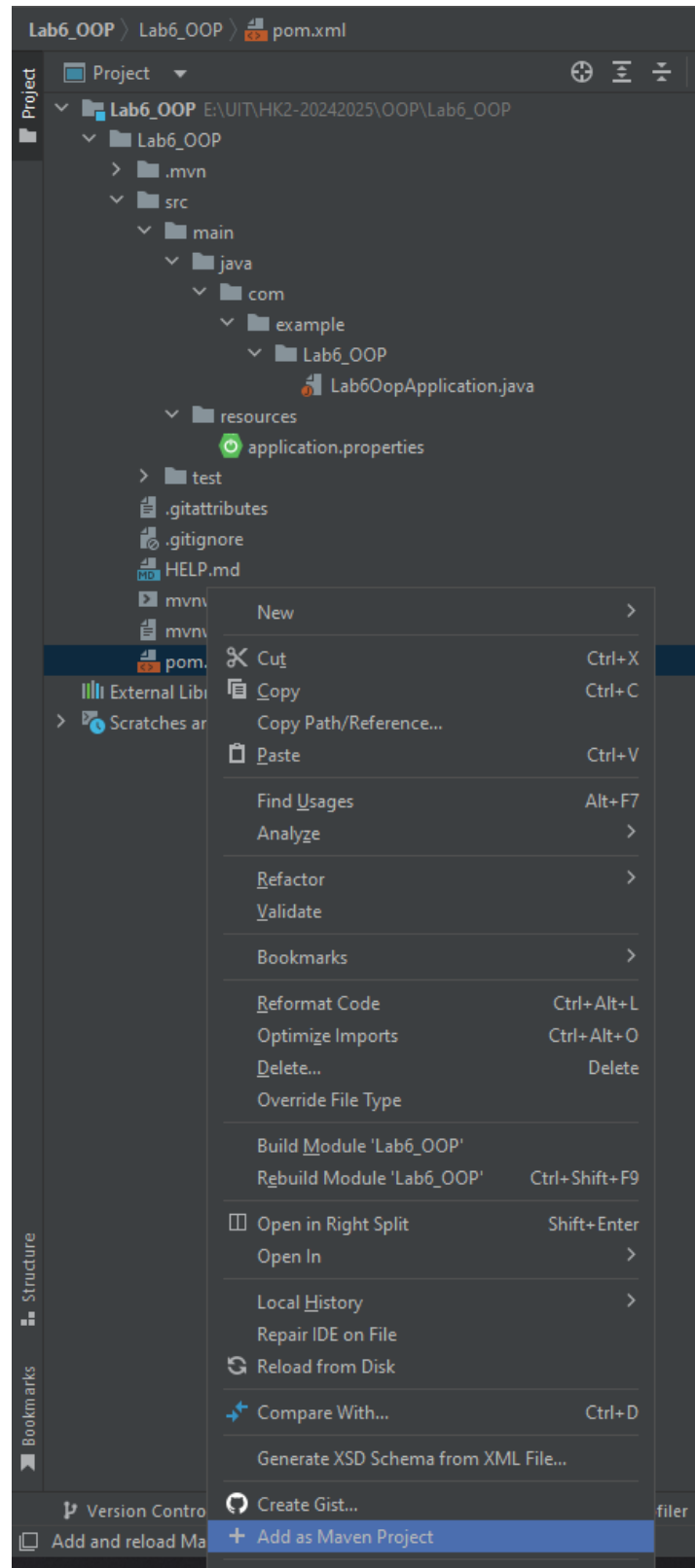
Lab6\_OOP.zip



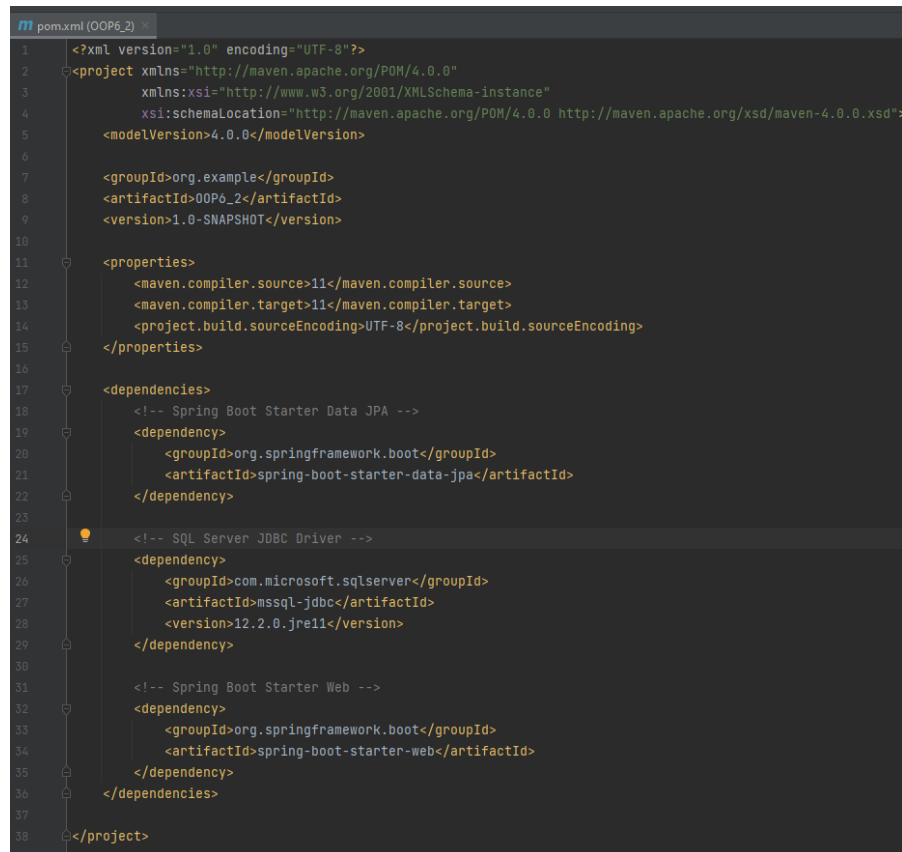
Lab6\_OOP

3. Open the extracted folder with IntelliJ.

4. In the pom.xml file, select "Add as Maven Project":



5. Open the pom.xml file and add the following code to include dependencies for JPA, JDBC Driver, and Starter Web in the project:



```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
5   <modelVersion>4.0.0</modelVersion>
6
7   <groupId>org.example</groupId>
8   <artifactId>OOP6_2</artifactId>
9   <version>1.0-SNAPSHOT</version>
10
11   <properties>
12     <maven.compiler.source>11</maven.compiler.source>
13     <maven.compiler.target>11</maven.compiler.target>
14     <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
15   </properties>
16
17   <dependencies>
18     <!-- Spring Boot Starter Data JPA -->
19     <dependency>
20       <groupId>org.springframework.boot</groupId>
21       <artifactId>spring-boot-starter-data-jpa</artifactId>
22     </dependency>
23
24     <!-- SQL Server JDBC Driver -->
25     <dependency>
26       <groupId>com.microsoft.sqlserver</groupId>
27       <artifactId>mssql-jdbc</artifactId>
28       <version>12.2.0.jre11</version>
29     </dependency>
30
31     <!-- Spring Boot Starter Web -->
32     <dependency>
33       <groupId>org.springframework.boot</groupId>
34       <artifactId>spring-boot-starter-web</artifactId>
35     </dependency>
36   </dependencies>
37
38 </project>
```

Code:

```
<!-- Spring Boot Starter Data JPA -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

```
<!-- SQL Server JDBC Driver -->
<dependency>
  <groupId>com.microsoft.sqlserver</groupId>
  <artifactId>mssql-jdbc</artifactId>
  <version>12.2.0.jre11</version>
</dependency>
```

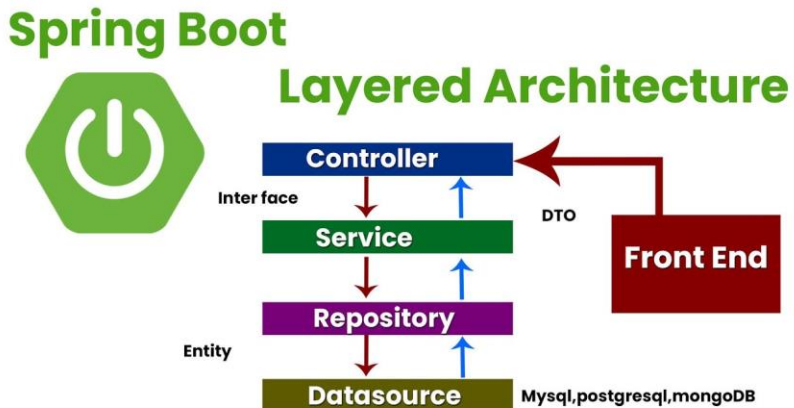
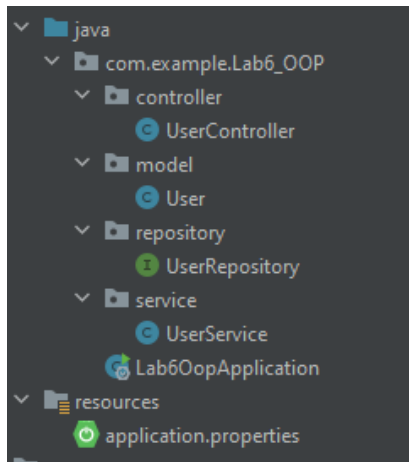
```
<!-- Spring Boot Starter Web -->
```

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>

```

6. Create folders according to the directory structure shown below:



File **UserController.java**:

```

package com.example.Lab6_OOP.controller;

import com.example.Lab6_OOP.model.User;
import com.example.Lab6_OOP.service.UserService;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/users")
public class UserController {
    private final UserService service;

    public UserController(UserService service) { this.service = service; }

    @PostMapping
    public User insert(@RequestBody User user) { return service.insert(user); }

    @PutMapping("/{id}")
    public User update(@PathVariable int id, @RequestBody User user) {
        user.setId(id);
        return service.update(user);
    }

    @DeleteMapping("/{id}")
    public void delete(@PathVariable int id) { service.delete(id); }
}

```

File **User.java**:

```
package com.example.Lab6_00P.model;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "Users")
public class User {

    @Id
    private int id;

    private String name;

    // Getters and setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}
```

File **UserRepository.java**:

```
package com.example.Lab6_00P.repository;

import com.example.Lab6_00P.model.User;
import org.springframework.data.jpa.repository.JpaRepository;

public interface UserRepository extends JpaRepository<User, Integer> {
}
```

### File UserService.java:

```
package com.example.Lab6_OOP.service;

import com.example.Lab6_OOP.model.User;
import com.example.Lab6_OOP.repository.UserRepository;
import org.springframework.stereotype.Service;

import java.util.Optional;

@Service
public class UserService {

    private final UserRepository repository;

    public UserService(UserRepository repository) { this.repository = repository; }

    public User insert(User user) { return repository.save(user); }

    public User update(User user) { return repository.save(user); // same method for insert/update }

    public void delete(int id) { repository.deleteById(id); }

    public Optional<User> findById(int id) { return repository.findById(id); }
}
```

### 7. Configure the application.properties file as shown below

```
application.properties
1  spring.application.name=Lab6_OOP
2  spring.application.port=8080
3  spring.datasource.url=jdbc:sqlserver://localhost:50075;databaseName=Lab6_OOP;encrypt=true;trustServerCertificate=true
4  spring.datasource.username=sa
5  spring.datasource.password=123456
6  logging.level.com.microsoft.sqlserver.jdbc=ERROR
7
8
9  spring.jpa.hibernate.ddl-auto=update
10 spring.jpa.show-sql=true
11 spring.jpa.database-platform=org.hibernate.dialect.SQLServerDialect
12
```

### Code:

```
spring.application.name=Lab6_OOP
spring.application.port=8080
spring.datasource.url=jdbc:sqlserver://localhost:50075;databaseName=Lab6_OOP;enc
rypt=true;trustServerCertificate=true
spring.datasource.username=sa
spring.datasource.password=123456
logging.level.com.microsoft.sqlserver.jdbc=ERROR
```

*spring.jpa.hibernate.ddl-auto=update*

*spring.jpa.show-sql=true*

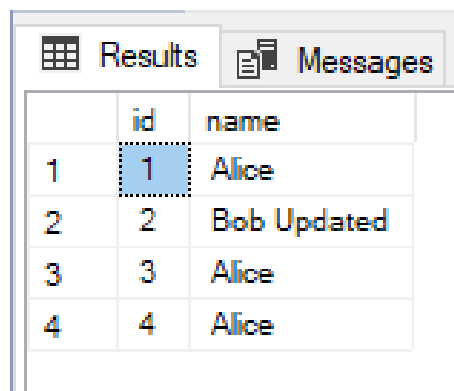
*spring.jpa.database-platform=org.hibernate.dialect.SQLServerDialect*

To test adding data, open Command Prompt and run the following command

```
C:\Users\THANH BINH>curl -X POST http://localhost:8080/users -H "Content-Type: application/json" -d '{"id":3,"name":"Alice"}'
{"id":3,"name":"Alice"}
C:\Users\THANH BINH>curl -X POST http://localhost:8080/users -H "Content-Type: application/json" -d '{"id":4,"name":"Alice"}'
{"id":4,"name":"Alice"}
C:\Users\THANH BINH>
```

CMD:

```
curl -X POST http://localhost:8080/users -H "Content-Type: application/json" -d
{"id":3,"name":"Alice"}
```



|   | id | name        |
|---|----|-------------|
| 1 | 1  | Alice       |
| 2 | 2  | Bob Updated |
| 3 | 3  | Alice       |
| 4 | 4  | Alice       |

Using **ORM (Object-Relational Mapping)** offers several advantages, especially in applications where you interact with a relational database (like SQL Server, MySQL, PostgreSQL) through an object-oriented programming language like Java.

**Key benefits:**

### 1. Abstraction from SQL

- You don't need to write SQL queries manually for basic operations like insert, update, delete, or select.
- Example: `userRepository.save(user)` instead of writing `INSERT INTO users ...`

### 2. Code Readability and Maintainability

- ORM maps database tables to classes, and rows to objects. You work with objects and fields instead of tables and columns.

- Code is cleaner and easier to understand, especially for those who know OOP but not SQL.

### **3. Automatic Table Creation and Schema Management**

- Frameworks like Hibernate (used in Spring Boot) can auto-generate tables from Java classes using annotations.
- Supports `ddl-auto=update`, which updates the database schema automatically when your model changes.

### **4. Type Safety**

- ORM helps you avoid common mistakes like SQL injection by using parameterized queries under the hood.
- You also benefit from compile-time checking of entity field names.

### **5. Reusability and Modularity**

- Repositories or DAO layers are reusable across the application.
- You can easily switch databases (e.g., from SQL Server to MySQL) with minimal code changes.

### **6. Built-in Caching**

- Most ORM tools have built-in caching mechanisms which can reduce database load and improve performance.

### **7. Relationships Management**

- ORM makes it easy to work with table relationships (like One-to-Many, Many-to-Many) using annotations like `@OneToMany`, `@ManyToOne`.

### **8. Better Testability**

- Because of the abstraction, you can mock repositories during unit testing without dealing with real database connections.

### **9. Transaction Management**

- Spring + ORM (like Hibernate) can manage transactions automatically, helping to maintain consistency in your data.

## II. SOLID principles

### 1. Single Responsibility Principle (SRP)

#### Definition:

A class should only have one reason to change, meaning it should only have one job or responsibility.

#### Why It Matters:

When a class does too much, even a small change can introduce bugs in unrelated functionality. SRP encourages cohesion—a class focused on a single task.

Bad Code (Violates SRP):

Report handles **3 responsibilities**: generating, printing, saving.

```
class Report {  
    public String generate() {  
        return "Report data...";  
    }  
  
    public void print(String data) {  
        System.out.println("Printing: " + data);  
    }  
  
    public void save(String data) {  
        System.out.println("Saving to DB: " + data);  
    }  
}
```

Refactored Code (Follows SRP):

#### Problem Solved:

- Changes in printing/saving logic won't affect report generation
- Easier to test and maintain

```
class Report {
    public String generate() {
        return "Report data...";
    }
}

class ReportPrinter {
    public void print(String data) {
        System.out.println("Printing: " + data);
    }
}

class ReportSaver {
    public void save(String data) {
        System.out.println("Saving to DB: " + data);
    }
}
```

## 2. Open/Closed Principle (OCP)

### Definition:

Software entities (classes, modules, functions) should be open for extension, but closed for modification.

### Why It Matters:

You want to extend functionality without touching existing code, reducing the risk of breaking existing systems.

Bad Code (Violates OCP): Modifying the class every time a new payment method is added

```

class PaymentProcessor {
    public void process(String type) {
        if (type.equals("credit")) {
            System.out.println("Processing credit payment");
        } else if (type.equals("paypal")) {
            System.out.println("Processing PayPal payment");
        }
    }
}

```

Refactored Code (Follows OCP):

```

2 implementations
interface Payment {
    2 implementations
    void process();
}

class CreditPayment implements Payment {
    public void process() {
        System.out.println("Processing credit payment");
    }
}

class PayPalPayment implements Payment {
    public void process() {
        System.out.println("Processing PayPal payment");
    }
}

class PaymentProcessor {
    public void process(Payment payment) {
        payment.process();
    }
}

```

### Problem Solved:

- Easily add new payment methods without changing PaymentProcessor
- Avoid regression bugs by not altering existing logic

### 3. Liskov Substitution Principle (LSP)

#### Definition:

Objects of a subclass should be able to replace objects of the superclass without altering the correctness of the program.

#### Why It Matters:

This ensures polymorphism works correctly. A derived class shouldn't break behavior expected from the base class.

Bad Code (Violates LSP): Ostrich violates the contract of Bird — can't substitute without error.

```
1 inheritor
class Bird {
    1 override
    public void fly() {
        System.out.println("Flying");
    }
}

class Ostrich extends Bird {
    public void fly() {
        throw new UnsupportedOperationException("Ostriches can't fly!");
    }
}
```

Refactored Code (Follows LSP):

```
3 implementations
interface Bird {}

1 implementation
interface FlyingBird extends Bird {
    1 implementation
    void fly();
}

class Sparrow implements FlyingBird {
    public void fly() {
        System.out.println("Sparrow flying");
    }
}

class Ostrich implements Bird {
    // Ostrich doesn't implement fly - no contradiction
}
```

### Problem Solved:

- Prevents run-time errors due to invalid subclass behavior
- Enforces correct usage of inheritance/polymorphism

## 4. Interface Segregation Principle (ISP)

### Definition:

Clients should not be forced to implement interfaces they don't use.

### Why It Matters:

Avoid creating "fat" interfaces. Let classes depend only on what they actually need.

Bad Code (Violates ISP): Robot is forced to implement an irrelevant method.

```
1 implementation
interface Worker {
    1 implementation
    void work();
    1 implementation
    void eat();
}

class Robot implements Worker {
    public void work() {}
    public void eat() {
        throw new UnsupportedOperationException("Robots don't eat");
    }
}
```

Refactored Code (Follows ISP):

```
2 implementations
interface Workable {
    2 implementations
    void work();
}

1 implementation
interface Eatable {
    1 implementation
    void eat();
}
```

```

class Human implements Workable, Eatable {
    public void work() {}
    public void eat() {}
}

class Robot implements Workable {
    public void work() {}
}

```

### Problem Solved:

- Prevents meaningless or empty implementations
- Promotes clean, focused interfaces

## 5. Dependency Inversion Principle (DIP)

### Definition:

High-level modules should not depend on low-level modules. Both should depend on abstractions.

### Why It Matters:

Your main logic shouldn't be tightly coupled to specific implementations (e.g., MySQL, REST, etc.). This allows swapping, mocking, and testing more easily.

Bad Code (Violates DIP): High-level module (App) depends on low-level concrete (MySQLDatabase).

```

class MySQLDatabase {
    public void connect() {
        System.out.println("Connected to MySQL");
    }
}

class App {
    private MySQLDatabase db = new MySQLDatabase();

    public void run() {
        db.connect();
    }
}

```

Refactored Code (Follows DIP):

```
1 implementation
interface Database {
    1 implementation
    void connect();
}

class MySQLDatabase implements Database {
    public void connect() {
        System.out.println("Connected to MySQL");
    }
}

class App {
    private Database db;

    public App(Database db) {
        this.db = db;
    }

    public void run() {
        db.connect();
    }
}
```

#### Problem Solved:

- Easily switch databases (e.g., SQLite, Postgres)
- Enables dependency injection, better testing, and flexibility.

### III. Design pattern

#### What is a Design Pattern?

A **design pattern** is a **reusable solution** to a common problem that occurs in software design. It is **not code**, but a **template or guide** on how to solve a particular design issue in different situations.

Think of it like a **blueprint** for solving software problems that developers have encountered many times before.

#### Key Characteristics of Design Patterns:

- **Reusable:** Can be applied to multiple projects.

- **Well-tested:** Based on real-world software development experience.
- **Standardized:** Shared vocabulary for developers (e.g., Singleton, Factory, Strategy).
- **Maintainable:** Promotes cleaner, more organized, and flexible code.

## Categories of Design Patterns

### 1. Creational Patterns

- Focus on how objects are created.
- Examples: Singleton, Factory, Builder, Prototype

### 2. Structural Patterns

- Focus on how objects are composed to form larger structures.
- Examples: Adapter, Decorator, Composite, Facade

### 3. Behavioral Patterns

- Focus on communication between objects.
- Examples: Strategy, Observer, Command, State

## Why Use Design Patterns?

- Makes your code **more readable** and **understandable** to other developers.
- Helps you **avoid reinventing the wheel**.
- Leads to **flexible**, **maintainable**, and **scalable** software.
- Supports **SOLID principles** and **best practices** in object-oriented design.

## 1. Singleton Pattern

### Concept:

Ensures that a class has only one instance and provides a global point of access to it. It's useful for managing shared resources like configuration, logging, database connections, etc.

Without Singleton:

```
public class Logger {
    public void log(String message) {
        System.out.println("Log: " + message);
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Logger logger1 = new Logger();
        Logger logger2 = new Logger();

        logger1.log("Hello");
        logger2.log("World");
    }
}
```

### Disadvantages:

- Multiple instances = inconsistent logging.
- Wastes memory and resources.
- Hard to manage and synchronize shared state.

With Singleton:

```
public class Logger {
    private static Logger instance;

    private Logger() {} // private constructor

    public static Logger getInstance() {
        if (instance == null) {
            instance = new Logger();
        }
        return instance;
    }

    public void log(String message) {
        System.out.println("Log: " + message);
    }
}
```

```

public class Main {
    public static void main(String[] args) {
        Logger logger1 = Logger.getInstance();
        Logger logger2 = Logger.getInstance();

        logger1.log("Hello");
        logger2.log("World");

        System.out.println(logger1 == logger2); // true
    }
}

```

### Advantages:

- Only one instance shared across the app.
- Saves memory, easy to manage global state.
- Ensures consistency (e.g., in logging or config access).

## 2. Strategy Pattern

Concept:

Defines a family of algorithms, encapsulates each one, and makes them interchangeable. Useful when you want to switch between algorithms/behaviors at runtime.

Without Strategy:

```

public class PaymentService {
    public void pay(String method) {
        if (method.equals("credit")) {
            System.out.println("Paid with credit card");
        } else if (method.equals("paypal")) {
            System.out.println("Paid with PayPal");
        }
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        PaymentService payment = new PaymentService();
        payment.pay( method: "credit");
        payment.pay( method: "paypal");
    }
}

```

## Disadvantages:

- Hard to extend: adding new payment methods means modifying the existing class.
- Violates **Open/Closed Principle** (should be open for extension, closed for modification).
- Difficult to test each payment method independently.

## With Strategy:

```
1 related problem
interface PaymentStrategy {
    2 implementations 2 related problems
    void pay();
}

class CreditCardPayment implements PaymentStrategy {
    2 related problems
    public void pay() {
        System.out.println("Paid with credit card");
    }
}

class PayPalPayment implements PaymentStrategy {
    2 related problems
    public void pay() {
        System.out.println("Paid with PayPal");
    }
}

1 related problem
class PaymentService {
    private PaymentStrategy strategy;

    1 related problem
    public PaymentService(PaymentStrategy strategy) {
        this.strategy = strategy;
    }

    public void executePayment() {
        strategy.pay();
    }
}
```

```
public class Main {  
    public static void main(String[] args) {  
        PaymentService creditPayment = new PaymentService(new CreditCardPayment());  
        PaymentService paypalPayment = new PaymentService(new PayPalPayment());  
  
        creditPayment.executePayment();  
        paypalPayment.executePayment();  
    }  
}
```

### Advantages:

- Easy to add new payment methods (just create a new class).
- Each strategy is testable in isolation.
- Encourages **code reuse** and **clean architecture**.

**Note:** To learn more about design patterns, you can read the book “*Head First Design Patterns*” (Java).

